

Message Broker + Django Redis, Celery & Channels Setup

Part 1 -- Message Broker

1. What is a Message Broker?

A **Message Broker** is middleware that helps applications or services **send, receive, store, and deliver messages** between each other.

Simply:

> A Message Broker is a **message delivery system** between applications or services.

Basic architecture:

``text

Producer

Message Broker

Consumer

``

The producer and consumer do not necessarily need to communicate directly.

2. Producer and Consumer

Producer

The **Producer** creates and sends a message.

Example:

```
```text
```

Django Application

"User placed an order"

```
```
```

Here, the Django application is the producer.

Consumer

The **Consumer** receives and processes the message.

Example:

```
```text
```

"User placed an order"

Payment Worker

```
```
```

Here, the payment worker is the consumer.

```
---
```

3. What Does a Message Broker Do?

A message broker can provide several messaging functions depending on the broker and configuration.

1. Receive Messages

```
```text
```

Producer

Broker

```
```
```

The broker receives messages from producers.

2. Queue or Retain Messages

If a consumer is temporarily unavailable or busy, messages can be retained according to the broker's delivery model.

```text

Producer

Broker

Queue

Consumer

```

3. Deliver Messages

The broker delivers messages to the appropriate consumer or consumers according to the messaging pattern.

4. Reliability Features

Depending on the broker and configuration, features may include:

```text

Acknowledgements

Retries

Persistence

Dead-letter handling

Message ordering

Delivery guarantees

---

These features are **not identical across all messaging systems**.

---

## # 4. Why Do We Use a Message Broker?

### ## 4.1 Decoupling

Without a broker:

```text

Django Email Service

The Django application directly depends on the email service.

With a broker:

```text

Django

Message Broker

Email Worker

Email Service

---

This creates a more loosely coupled architecture.

---

## # 5. Asynchronous Processing

One of the most common reasons for using a message broker is **asynchronous**

---

processing\*\*.

Example:

```text

User Signup

Django

"Send Welcome Email"

Broker

Email Worker

```

The Django request does not have to perform the email operation itself.

Therefore, a slow background operation does not necessarily block the user's request.

---

## # 6. Scalability

Suppose one worker cannot process all incoming tasks.

You can add multiple workers:

```text

Worker 1

Producer Broker Worker 2

Worker 3

```

Multiple workers can process work concurrently, depending on the workload and

---

configuration.

---

## # 7. Common Message Brokers and Messaging Systems

### ## RabbitMQ

RabbitMQ is a **general-purpose message broker**.

Common concepts include:

```text`

Queues

Exchanges

Routing

Acknowledgements

Retries

Dead-lettering

`````

It is widely used for asynchronous communication and task queues.

---

### ## Redis

Redis is primarily an **in-memory data store**, but it also provides messaging-related capabilities such as:

```text`

Pub/Sub

Streams

Lists

...

Redis can also be used as a broker or result backend by systems such as Celery.

Important:

> Redis is not simply synonymous with "message broker." Its role depends on which Redis feature and application architecture are being used.

Apache Kafka

Kafka is a **distributed event streaming platform** designed for high-throughput and durable event streams.

Common use cases include:

``text

Event streaming

Log pipelines

Data integration

Real-time analytics

Large-scale event processing

...

Kafka's architecture and semantics differ from traditional queue-oriented brokers such as RabbitMQ.

8. Message Broker vs Task Queue

These concepts are related but not identical.

Message Broker

A message broker transports messages between systems.

```text

Producer

Broker

Consumer

```

Task Queue

A task queue is used to distribute background jobs to workers.

```text

Application

Task Queue

Worker

```

Celery is a distributed task queue system that commonly uses a broker such as Redis or RabbitMQ.

9. Real-Life Examples

Example 1 -- Comment Moderation

```text

User submits comment

Django

Message Broker

Moderation Worker

...

The moderation task can run in the background.

## Example 2 -- E-commerce

```text

Order Placed

Broker

/\

Payment Inventory

Worker Worker

...

Different consumers can process different parts of the workflow.

Example 3 -- Email

```text

User Signup

Django

Broker

Email Worker

Welcome Email

---

---

## # Part 2 -- Django + Redis Setup on Windows

There are two common development approaches:

### ### Option A -- Windows + WSL

```text

Windows

WSL

Ubuntu

Python + Django + Redis

Option B -- Native Windows

```text

Windows

Python + Django

Redis for Windows-compatible setup

---

For the native Windows setup below, Redis must be installed and running using a Windows-compatible Redis distribution/environment.

---

## # Part 3 -- Django + Redis + Celery + Channels

Our application can use Redis for several different purposes:

```
```text
```

Django

Cache Celery Channels

Redis

```
```
```

These are different use cases even when they use the same Redis server.

```

```

# Part 4 -- Install Python Packages

Create and activate a virtual environment first.

```
```bash
```

```
python -m venv .venv
```

```
```
```

Activate on Windows:

```
```powershell
```

```
.venv\Scripts\activate
```

```
```
```

Then install the required packages:

```
```bash
```

```
python -m pip install django celery redis channels channels-redis
```

```

If Django Redis cache is also required:

```
```bash
python -m pip install django-redis
```

```

A complete installation can therefore be:

```
```bash
python -m pip install django celery redis django-redis channels channels-redis
```

```

---

## # Part 5 -- Create Django Project

Create the project:

```
```bash
django-admin startproject config
```

```

Move into the project:

```
```bash
cd config
```

```

Create an application:

```
```bash
python manage.py startapp account
```

...

Example structure:

```text

project/

manage.py

config/

\_\_init\_\_.py

settings.py

urls.py

asgi.py

wsgi.py

celery.py

account/

migrations/

\_\_init\_\_.py

admin.py

apps.py

models.py

tasks.py

tests.py

views.py

...

---

# Part 6 -- Celery Configuration

Create:

```
```text  
config/celery.py  
```
```

Add:

```
```python  
import os  
  
from celery import Celery  
  
os.environ.setdefault(  
    "DJANGO_SETTINGS_MODULE",  
    "config.settings",  
)  
  
app = Celery("config")  
  
app.config_from_object(  
    "django.conf:settings",  
    namespace="CELERY",  
)  
  
app.autodiscover_tasks()  
```  

Part 7 -- Load Celery When Django Starts
```

It is common to import the Celery application from:

```
```text
config/__init__.py
```
```

Add:

```
```python
from .celery import app as celery_app

__all__ = ("celery_app",)
```
```

This helps ensure the Celery app is loaded with the Django project.

---

# Part 8 -- Configure Celery + Redis

In:

```
```text
config/settings.py
```
```

add:

```
```python
CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"

CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/0"
```
```

Architecture:

---

```
```text
```

Django

Celery

Redis

Celery Worker

```
```
```

```
Broker
```

```
```python
```

```
CELERY_BROKER_URL
```

```
```
```

The broker is used to transport tasks from the application to workers.

```
Result Backend
```

```
```python
```

```
CELERY_RESULT_BACKEND
```

```
```
```

This can store task results when result storage is enabled and configured.

```

```

```
Part 9 -- Create a Celery Task
```

Create:

```
```text
```

```
account/tasks.py
```

...

Example:

```
```python
from celery import shared_task

@shared_task
def add(x, y):
 return x + y
...

```

Another example:

```
```python
from celery import shared_task

@shared_task
def send_otp_email(email, otp):
    # Email sending logic
    print(f'Sending OTP {otp} to {email}')

    return True
...

```

Part 10 -- Install and Run Redis on Windows

Redis needs to be running before Celery can communicate with it.

For a native Windows environment, use a supported Windows-compatible Redis distribution.

If the Redis executable is available directly, you may run:

```
```powershell
redis-server.exe
```
```

Then open another terminal and test:

```
```powershell
redis-cli.exe ping
```
```

Expected:

```
```text
PONG
```
```

If your Redis installation provides the `redis-cli` command directly, you can use:

```
```powershell
redis-cli ping
```
```

Part 11 -- Run Celery Worker on Windows

Open another terminal.

Activate the virtual environment:

```
```powershell
.venv\Scripts\activate
```

```

Run:

```
```powershell
celery -A config worker --loglevel=info --pool=solo
```
```

Why `--pool=solo`?

The `solo` pool runs tasks in the worker's main process and is commonly useful for local development on Windows.

The exact worker pool choice depends on the Celery version and deployment environment.

Part 12 -- Run Django

Open another terminal.

Activate the virtual environment:

```
```powershell
.venv\Scripts\activate
```
```

Run migrations:

```
```powershell
python manage.py migrate
```
```

Start Django:

```
```powershell
```

```
python manage.py runserver
```

```
```
```

Open:

```
```text
```

```
http://127.0.0.1:8000/
```

```
```
```

```
---
```

Part 13 -- Test Celery Task

Start Django shell:

```
```powershell
```

```
python manage.py shell
```

```
```
```

Then:

```
```python
```

```
from account.tasks import add
```

```
result = add.delay(10, 20)
```

```
```
```

`delay()` sends the task asynchronously.

To retrieve the result:

```
```python
```

```
result.get()
```

...

Expected:

```
```text
```

30

...

For a web request, avoid unnecessarily waiting for asynchronous task results because that removes much of the benefit of background processing.

Part 14 -- Test OTP Task

If you have:

```
```python
```

```
from celery import shared_task
```

```
@shared_task
```

```
def send_otp_email(email, otp):
```

```
 print(f"Sending OTP {otp} to {email}")
```

```
 return True
```

```
```
```

You can test:

```
```python
```

```
from account.tasks import send_otp_email
```

```
result = send_otp_email.delay(
```

```
"test@mail.com",
```

```
"123456",
```

```
)
```

```
...
```

The task is sent to Redis and picked up by the Celery worker.

Architecture:

```
```text
```

Django Shell

```
send_otp_email.delay()
```

Celery

Redis Broker

Celery Worker

```
send_otp_email()
```

```
...
```

```
---
```

Part 15 -- Django Channels + Redis

What is Django Channels?

Django Channels extends Django's capabilities beyond traditional HTTP request/response handling.

It is commonly used for:

```text

WebSockets

Real-time communication

Chat applications

Live notifications

Real-time updates

```

Architecture:

```text

Browser

WebSocket

Django Channels

Redis Channel Layer

Other Consumers

```

Part 16 -- Configure ASGI

In:

```text

config/settings.py

```

add:

```
```python
ASGI_APPLICATION = "config.asgi.application"
```
```

The project also needs an appropriate `asgi.py`, routing, and WebSocket consumer configuration.

Simply installing Channels and configuring `CHANNEL_LAYERS` does **not** create a complete WebSocket application.

Part 17 -- Configure Redis Channel Layer

In:

```
```text
config/settings.py
```
```

add:

```
```python
CHANNEL_LAYERS = {
 "default": {
 "BACKEND": "channels_redis.core.RedisChannelLayer",
 "CONFIG": {
 "hosts": [
 ("127.0.0.1", 6379),
],
 },
 },
}
```

```
}
...


```

The Redis server provides the channel layer used by Channels for communication between consumers/processes.

```

```

## # Part 18 -- Django Redis Cache

If Django caching is required, install:

```
```bash  
python -m pip install django-redis  
...  

```

Then configure:

```
```python  
CACHES = {
 "default": {
 "BACKEND": "django_redis.cache.RedisCache",
 "LOCATION": "redis://127.0.0.1:6379/1",
 "OPTIONS": {
 "CLIENT_CLASS": "django_redis.client.DefaultClient",
 },
 },
}
...

```

Here:

```
```text
```

127.0.0.1

Redis server

6379

Redis port

/1

Redis logical database 1

```
```
```

```

```

## # Part 19 -- Redis Database Separation

A simple development setup can use:

```
```text
```

Redis

DB 0 Celery

DB 1 Django Cache

```
```
```

Example:

```
```python
```

```
CELERY_BROKER_URL = (
```

```
"redis://127.0.0.1:6379/0"
```

```
)
```

```
```
```

and:

```
```python
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
    }
}
```
```

The `0` and `1` represent Redis logical database numbers.

For production, stronger isolation may be preferable depending on the architecture, such as separate Redis instances, services, or appropriate key namespaces.

---

#### # Part 20 -- Complete `settings.py` Example

A simplified configuration can look like:

```
```python
# Celery

CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"

CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/0"

# Channels

ASGI_APPLICATION = "config.asgi.application"

CHANNEL_LAYERS = {
```

```
"default": {  
    "BACKEND": "channels_redis.core.RedisChannelLayer",  
    "CONFIG": {  
        "hosts": [  
            ("127.0.0.1", 6379),  
        ],  
    },  
}
```

```
# Django Cache
```

```
CACHES = {  
    "default": {  
        "BACKEND": "django_redis.cache.RedisCache",  
        "LOCATION": "redis://127.0.0.1:6379/1",  
        "OPTIONS": {  
            "CLIENT_CLASS": "django_redis.client.DefaultClient",  
        },  
    },  
}  
...
```

```
---
```

```
# Part 21 -- Complete Windows Development Workflow
```

Open separate terminals.

```
## Terminal 1 -- Redis
```

```
```powershell
redis-server.exe
```
```

Check:

```
```powershell
redis-cli.exe ping
```
```

Expected:

```
```text
PONG
```
```

Terminal 2 -- Celery

```
```powershell
.venv\Scripts\activate
```
```

Then:

```
```powershell
celery -A config worker --loglevel=info --pool=solo
```
```

Terminal 3 -- Django

```
```powershell
.venv\Scripts\activate
```
```

Then:

```
```powershell
python manage.py migrate
python manage.py runserver
```
```

Terminal 4 -- Testing

```
```powershell
.venv\Scripts\activate
```
```

Then:

```
```powershell
python manage.py shell
```
```

Test:

```
```python
from account.tasks import add

result = add.delay(10, 20)

result.get()
```
```

...

Expected:

```text

30

...

---

# Part 22 -- Final Architecture

```text

Django

Cache Celery Channels

Redis Redis Channel

Broker Layer

Celery Worker WebSocket

Redis

...

A simpler mental model:

```text

Django

## Cache Celery Channels

Redis

Celery Worker WebSocket

...

---

# Part 23 -- Common Mistakes

## Mistake 1 -- Redis is not running

If Redis is unavailable, Celery and Redis-based features cannot communicate with it.

Test:

```
```powershell
```

```
redis-cli.exe ping
```

```
```
```

Expected:

```
```text
```

```
PONG
```

```
```
```

---

## Mistake 2 -- Celery worker is not running

Installing Celery is not enough.

You also need:

```
```powershell
celery -A config worker --loglevel=info --pool=solo
```
```

---

## Mistake 3 -- Installing packages globally

Avoid relying on a global Python environment.

Prefer:

```
```powershell
python -m venv .venv
.venv\Scripts\activate
python -m pip install django celery redis channels channels-redis
```
```

---

## Mistake 4 -- Forgetting `celery.py`

Make sure the project contains:

```
```text
config/
celery.py
```
```

and:

```
```python
app = Celery("config")
```

```
---
```

```
---
```

Mistake 5 -- Forgetting Celery initialization

In:

```
```text
```

```
config/__init__.py
```

```

```

use:

```
```python
```

```
from .celery import app as celery_app
```

```
__all__ = ("celery_app",)
```

```
---
```

```
---
```

Mistake 6 -- Wrong Django settings module

For this project:

```
```python
```

```
os.environ.setdefault(
```

```
"DJANGO_SETTINGS_MODULE",
```

```
"config.settings",
```

```
)
```

```

```

The project package name must match your actual Django project.

---

## ## Mistake 7 -- Confusing Cache with Celery

Django Cache:

```text

Django

Redis

Cached Data

```

Celery:

```text

Django

Celery Task

Redis Broker

Worker

```

They use Redis for completely different purposes.

---

## # Part 24 -- Quick Revision

### ## Message Broker

```text

Producer

Message Broker

Consumer

...

Purpose:

``text

Message delivery

Decoupling

Asynchronous processing

Scalability

Reliability features

...

RabbitMQ

``text

General-purpose message broker

...

Redis

``text

In-memory data store

+

Caching

+

Messaging capabilities

+

Celery broker/backend

+

Channels channel layer

...

Kafka

```text

Distributed event streaming platform

...

---

## Celery

```text

Distributed task queue

...

Typical architecture:

```text

Django

Celery

Redis/RabbitMQ

## Celery Worker

```

Django Cache

```python

from django.core.cache import cache

cache.set(

"key",

"value",

timeout=60,

)

value = cache.get("key")

```

Celery Broker

```python

CELERY\_BROKER\_URL = (

"redis://127.0.0.1:6379/0"

)

```

Django Channels

```

```python
CHANNEL_LAYERS = {
 "default": {
 "BACKEND": (
 "channels_redis.core.RedisChannelLayer"
),
 "CONFIG": {
 "hosts": [
 ("127.0.0.1", 6379),
],
 },
 },
}
```

```

Final Mental Model

```

```text

```

APPLICATION

Django Backend

Cache Celery Channels

Redis Redis

Broker Channel Layer

Cached Celery WebSocket

Data Worker Consumers

...

## One-Line Definitions

```text

Message Broker

Middleware that transports messages between producers and consumers.

Producer

Creates and sends messages.

Consumer

Receives and processes messages.

Redis

In-memory data store with caching and messaging capabilities.

Celery

Distributed task queue for background/asynchronous jobs.

RabbitMQ

General-purpose message broker.

Kafka

Distributed event streaming platform.

Django Cache

Stores reusable/temporary data to reduce expensive operations.

Django Channels

Extends Django for use cases such as WebSockets and real-time communication.

Redis Channel Layer

Allows Channels processes/consumers to communicate through Redis.

Celery Worker

A process that receives and executes Celery tasks.

...

Final Windows Commands -- Cheat Sheet

Create virtual environment

```
```powershell
```

```
python -m venv .venv
```

```
```
```

Activate

```
```powershell
```

```
.venv\Scripts\activate
```

```
```
```

Install packages

```
```powershell
```

```
python -m pip install django celery redis django-redis channels channels-redis
```

```
```
```

Run Redis

```
```powershell
```

```
redis-server.exe
```

```
...
```

```
Test Redis
```

```
```powershell
```

```
redis-cli.exe ping
```

```
...
```

Expected:

```
```text
```

```
PONG
```

```
...
```

```
Run Celery
```

```
```powershell
```

```
celery -A config worker --loglevel=info --pool=solo
```

```
...
```

```
#### Run Django
```

```
```powershell
```

```
python manage.py migrate
```

```
python manage.py runserver
```

```
...
```

```
Test Celery
```

```
```python
```

```
from account.tasks import add
```

```
result = add.delay(10, 20)
```

```
result.get()
```

```
...
```

Expected:

```
```text
```

```
30
```

```
...
```